

TP3

Local Filtering and Histograms

In this practical work, we study local filtering and image improvements through histogram modifications.

Exercise 1 : Filtering

1. Implement the function *binomial_filter* in `TP3/filters.c` that computes the image smoothed with the following 3×3 binomial filter one time:

$$\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} \quad (1)$$

Be careful with the boundary conditions. Try it on `boat_noise1.pgm`.

```
% ./smoothimage images/boat_noise1.pgm
                      results/boat_noise1_smooth3_1.pgm 3
```

2. Implement the option to smooth the image N times and run

```
% ./smoothimage images/boat_noise1.pgm
                      results/boat_noise1_smooth3_5.pgm 3 5
```

3. Implement the option to smooth with a filter of dimension 5×5 (binomial coefficients can be obtained from the Pascal triangle) and run

```
% ./smoothimage images/boat_noise1.pgm
                      results/boat_noise1_smooth5_5.pgm 5 5
```

We are now interested in using a median filter to smooth the image. For that we will use the program `medianfilter`:

4. Implement a median filter in the file `TP3/filters.c` and try it with

```
./medianfilter images/boat_noise1.pgm  
               results/boat_noise1_median3_1.pgm 3
```

5. Analogously to the binomial filter, implement the option to smooth the image N times and use a 5×5 window.

You implemented all filters. You can experiment with different input values.

6. Test other values for N and the filter sizes to explore the effects of all filters.
7. Try your new program with `boat_noise2.pgm` and other images.
8. Compare and discuss the results obtained with the binomial filters and the median filter with the different configurations and images.

Exercise 2 : Histograms

Image improvements through histogram modifications require first to compute the intensity histogram of an image. In `Utils/types.h` a structure of a histogram is provided (which is similar to a one dimensional image). In `TP3/histogram.h` a function `plot_histogram()` is given: it writes a CSV file with the histogram content.

1. Complete the program `createhistogram` by implementing the function `compute_histogram` in `TP3/histogram.c`. Test it by running

```
./createhistogram images/landscape.pgm  
                  results/landscape_histogram.csv
```

and plot the obtained histogram¹.

2. Try your new program with `images/moon.pgm` as well as other different images.

Now that you can compute histograms, in `TP3/histogram.h` you can:

3. Implement `histogram_stretching()` that computes the image transformed with a histogram stretching and test it with

```
./histogramstretching images/landscape.pgm  
                      results/landscape_stretched.pgm
```

4. Implement `histogram_equalization()` that computes the image transformed with a histogram equalization and test it with

¹You can plot the values of a CSV file using any external tool (excel, gnuplot, ...).

```
./histogramequalization images/landscape.pgm  
results/landscape_equalized.pgm
```

5. Compute and plot the new histograms of the transformed images and compare them to the original ones.
6. Try your new program with `images/moon.pgm` and other images.

Exercise 3 Bonus : Color Histograms

Extend the histogram programs to work on color (ppm) images².

1. Extend `createhistogram` to create a CSV with 3 channels.
2. Extend `histogramstretching` to stretch a 3-channel image.
3. Extend `histogramequalization` to equalize a 3-channel image.
4. What happens if you stretch / equalize each channel independently?
5. How could you fix the previous problem?

²What about using the TP1 functions in which you split the channels and computed a RGB to grayscale conversion?